

ALGORITMES IMPLEMENTATS

Assignatura: Projectes de Programació (PROP)

Projecte: Sistema de Clustering per a Anàlisi d'Enquestes

Autors: Grup 11.4

Data d'entrega: 17 de Novembre de 2025

Contenido

ALGORITMES IMPLEMENTATS	1
Resum del Document	1
1. K-Means	3
2. K-Means++	6
3. K-Medoids (PAM)	9
4. DistanceCalculator	12
5. ClusterEvaluator	15
6. Kluster	17
7. Taula Comparativa	20

Resum del Document

Aquest document descriu de manera exhaustiva els algorismes de clustering implementats en el nostre projecte. Per a cadascun dels algorismes, hem analitzat:

- **Estructura triada:** Les decisions de disseny preses per implementar l'algoritme
- **Justificació:** Per què hem escollit aquesta estructura i com funciona
- **Alternatives descartades:** Altres opcions que vam considerar i per què les hem rebutjat
- **Complexitat:** Anàlisi de la complexitat temporal i espacial

Els algorismes implementats són **K-Means**, **K-Means++** i **K-Medoids (PAM)**, a més de les classes auxiliars **DistanceCalculator**, **ClusterEvaluator** i **Kluster** que donen suport al sistema de clustering.

El document està pensat per explicar les nostres decisions d'implementació de manera que qualsevol persona del grup pugui entendre per què hem fet les coses d'una manera determinada i quines alternatives hem considerat.

1. K-Means

Descripció de l'Algoritme

K-Means és un algoritme de clustering que parteix el conjunt de dades en K grups (clusters) intentant minimitzar la distància entre cada punt i el centroide del seu cluster.

Com funciona:

1. **Inicialització:** Seleccionem K punts aleatoris del dataset com a centroides inicials
2. **Assignació:** Assignem cada punt al centroide més proper (segons distància Euclidiana)
3. **Actualització:** Recalculem els centroides com la mitjana de tots els punts assignats a cada cluster
4. **Convergència:** Repetim els passos 2-3 fins que els centroides no canviïn (o canviïn menys d'un llinard)

L'objectiu és minimitzar la **suma de distàncies al quadrat** (SSE - Sum of Squared Errors) entre cada punt i el seu centroide assignat.

Nota important sobre representants:

Els centroides calculats per K-Means són punts **artificials** (mitjanes, modes) que poden tenir valors impossibles en la realitat (per exemple, 27.33 anys). Per obtenir un participant real que representi cada cluster, es pot usar el mètode `getRepresentant()` que retorna el punt més proper al centroide.

Estructura Triada

Per implementar K-Means, hem triat una estructura basada en els següents components:

Classe principal: `KMeans.java`

Atributs clau:

```
private final DistanceCalculator dc = new DistanceCalculator();
private final Random rnd;
```

Estructures de dades: - `List<Kluster>` per emmagatzemar els K clusters - `String[]` per representar cada participant (array de respostes) - `DistanceCalculator` com a component per calcular distàncies

Justificació

Hem escollit aquesta estructura per diversos motius:

1. Per què `List<Kluster>` enlloc d'un array fix?

Inicialment vam considerar usar un array `Kluster[]`, però ens vam decantar per `List<Kluster>` perquè: - Ens dona més flexibilitat si en algun moment necessitem ajustar K dinàmicament - L'accés per índex segueix sent $O(1)$ - La interfície de `List` és més neta i facilita operacions com iterar sobre els clusters

2. Per què `String[]` per representar participants?

Aquesta va ser una de les decisions més importants. Cada participant té respostes a diferents preguntes que poden ser de tipus molt diversos (edat, satisfacció, colors preferits, comentaris...). Vam decidir representar-ho tot com a String perquè ens donava més flexibilitat ja que qualsevol dada es pot representar com a string, és més simple, ja que no hem d'anar convertint números a text i a l'invers i es compatible amb JSON (facilita la importació i exportació de dades)

3. Per què usar DistanceCalculator com a component separat?

Podríem haver posat tot el codi de càlcul de distàncies dins de K-Means, però vam preferir fer una classe separada perquè:

- És més ordenat: cada classe fa la seva feina
- Evitem repetir codi: els tres algoritmes comparteixen el mateix DistanceCalculator
- Podem testejar les distàncies sense haver d'executar el clustering complet

4. Per què injectar Random per constructor?

Això pot semblar estrany, però té sentit: - Permet fer tests amb una seed fixa per tenir resultats reproducibles - Evita crear múltiples instàncies de Random si executem K-Means varies vegades

Alternatives Descartades

Durant el disseny, vam considerar diverses alternatives que finalment vam descartar:

Alternativa	Per què la vam descartar
Object[] per participants	Massa overhead de memòria pel boxing/unboxing constant. Caldria fer casting a cada accés, cosa propensa a errors.
Map<String, Object> per participants	Un HashMap per cada participant consumeix molta memòria (buckets, factors de càrrega...). L'accés seria més lent que amb un array.
Array Kcluster[] fix	Massa rígida. Si volguéssim experimentar amb diferents valors de K o ajustar dinàmicament, seria un problema.
Centroides com a classe pròpia	Seria over-engineering. Els centroides són simplement vectors (String[]), no necessiten una classe separada.
Calcular distàncies dins de K-Means	Barrejaria responsabilitats. Si canviem la manera de calcular distàncies, hauríem de tocar K-Means.

Complexitat

Complexitat temporal:

L'algoritme té diverses fases, cada una amb la seva complexitat:

Inicialització:	$O(K)$	// Seleccionar K punts distints aleatoris
Assignació:	$O(N \times K \times D)$	// N punts $\times$ K clusters $\times$ D dimensions
Actualització:	$O(N \times D)$	// Recalcular K centroides
Per iteració:	$O(N \times K \times D)$	// El terme dominant
Total:	$O(I \times N \times K \times D)$	// I iteracions

On: - **I** = nombre d'iteracions fins convergència - **N** = nombre de participants - **K** = nombre de clusters - **D** = nombre de preguntes (dimensions)

Exemple pràctic:

Per un cas típic del nostre projecte: - N = 1.000 participants - K = 5 clusters - D = 20 preguntes - I $\approx$ 30 iteracions

Operacions totals $\approx 30 \times 1.000 \times 5 \times 20 = \mathbf{3.000.000 \text{ operacions}}$

En un ordinador modern ($\approx 10^9$ operacions/segon), això és aproximadament 3 mil·lisegons, totalment acceptable.

2. K-Means++

Descripció de l'Algoritme

K-Means++ és una millora de K-Means que utilitza una **inicialització intel·ligent** dels centroides. La diferència amb K-Means estàndard és només a la fase d'inicialització; després, el procés de clustering és idèntic.

Com funciona la inicialització intel·ligent:

1. **Primer centroide:** Seleccionem un punt completament aleatori del dataset
2. **Centroides següents:** Per cada centroide addicional (fins a K):
 - Per cada punt x (usuari que ha respost lenquesta), calculem $D(x)^2 =$ distància al quadrat al centroide més proper
 - Seleccionem el següent centroide aleatòriament amb probabilitat proporcional a $D(x)^2$
 - Els punts més llunyans tenen més probabilitat de ser escollits
3. **Clustering:** Un cop tenim els K centroides, executem K-Means normalment

Per què funciona millor:

El problema de K-Means normal és que pot escollir tots els centroides a l'atzar en una mateixa zona del dataset, i això fa que l'algoritme vagi més lent i doni pitjors resultats. K-Means++ reparteix els centroides de manera més intel·ligent des del principi, així que:

- Acabem amb millors grups (més qualitat)
- L'algoritme acaba més ràpid (menys iteracions)
- Evitem quedar-nos encallats en solucions dolentes

Nota sobre representants:

Igual que K-Means, els centroides de K-Means++ són punts calculats (mitjanes). Per obtenir participants reals que representin cada cluster, usar `getRepresentant()`.

Estructura Triada

K-Means++ és una millora de K-Means que només canvia la manera d'inicialitzar els centroides. L'estructura és:

Classe principal: `KMeansPlusPlus.java`

Atributs:

```
private final DistanceCalculator dc = new DistanceCalculator();
private final Random rnd;
```

Mètode clau: - `initCentroids()`: implementa la inicialització intel·ligent - `fit()`: delega a K-Means després de la inicialització

Justificació

1. Per què crear una classe separada enlloc d'afegir un paràmetre a K-Means?

Al principi vam pensar en afegir un paràmetre booleà a K-Means (tipus `useSmartInit``), però al final vam decidir crear una classe separada perquè:

- Cada classe fa una cosa: K-Means fa clustering, K-Means++ fa clustering amb millor inicialització
- És més clar: veus directament quin algoritme estàs usant
- Si demà volem afegir una altra manera d'inicialitzar, només cal crear una nova classe

2. Per què delegar a K-Means després de la inicialització?

Un cop tenim els centroides inicials, la resta és exactament igual que K-Means. No té sentit copiar tot el codi:

- Aprofitem el codi: cridem `KMeans.fitWithInitialCentroids()`
- Més fàcil de mantenir: si arreglem un bug a K-Means, K-Means++ també queda arreglat
- Funcionen igual: garantim que després de la inicialització, ambdós fan exactament el mateix

3. Com funciona la inicialització intel·ligent?

El truc de K-Means++ és escollir els centroides de manera intel·ligent:

1. Primer centroide: completament a l'atzar
2. Per cada centroide següent:
 - Calculem la distància al quadrat de cada punt al centroide més proper
 - Els punts més llunyans tenen més probabilitat de ser escollits com a següent centroide

Això fa que els centroides quedin ben repartits per tot l'espai.

4. Per què D^2 (distància al quadrat) i no simplement D ?

Aquesta decisió ve de l'article original de K-Means++. Elevar al quadrat fa que les diferències es facin més grans: un punt molt llunyà té molta més probabilitat de ser escollit que un de proper

Alternatives Descartades

Alternativa	Per què la vam descartar
Implementar K-Means++ complet	Duplicaria tot el codi de clustering. Si canviem alguna cosa, cal canviar-ho a dos llocs.
Usar D (distància lineal) enlloc de D^2	Sense les garanties teòriques de K-Means++. Els punts llunyans no tindrien prou prioritat.

Binary search per la ruleta	Més complex d'implementar i el guany de performance ($O(\log N)$ vs $O(N)$) és negligible per K petit.
------------------------------------	--

Complexitat

Complexitat temporal:

```

Inicialització K-Means++:  $O(K \times N \times D)$  // Seleccionar K centroides
Clustering K-Means:       $O(I \times N \times K \times D)$  // Igual que K-Means normal
Total:                    $O(K \times N \times D + I \times N \times K \times D)$ 
                        =  $O(I \times N \times K \times D)$  // Si  $I \gg K$ 

```

Nota important: Tot i que la inicialització té un cost extra de $O(K \times N \times D)$, en la pràctica K-Means++ convergeix més ràpid (menys iteracions I), així que el cost total sol ser similar o fins i tot menor que K-Means amb inicialització aleatòria.

3. K-Medoids (PAM)

Descripció de l'Algoritme

K-Medoids (també conegut com PAM - Partitioning Around Medoids) és similar a K-Means però amb una diferència fonamental: **els centroides són sempre punts reals del dataset** (medoides), no punts calculats com mitjanes.

Com funciona:

1. **Inicialització:** Seleccionem K punts aleatoris del dataset com a medoides inicials
2. **Assignació:** Assignem cada punt al medoide més proper (segons distància Manhattan)
3. **Actualització:** Per cada cluster:
 - Provem tots els punts del cluster com a candidats a nou medoide
 - Seleccionem el punt que minimitza la suma de distàncies dins del cluster
 - Si trobem un millor medoide, l'intercanviem
4. **Convergència:** Repetim els passos 2-3 fins que ni els medoides ni les assignacions canviïn

Diferències clau amb K-Means:

Aspecte	K-Means	K-Medoids
Centroides	Punts calculats (mitjanes)	Punts reals del dataset
Poden ser outliers?	Els centroides es veuen afectats	Els medoides són més robustos
Interpretabilitat	El centroide pot no ser un cas real	El medoide és un participant real
Cost computacional	$O(I \times N \times K \times D)$ - més ràpid	$O(I \times K \times N^2 \times D)$ - més lent

Per què és més robust a outliers:

Imagina un cluster de persones amb edats [20, 22, 23, 21, 90]: - K-Means calcularia centroide = $(20+22+23+21+90)/5 = 35.2$ (no representa ningú!) - K-Medoids seleccionaria un dels punts reals, probablement 22 o 23, ignorant l'outlier de 90

Nota sobre representants:

A diferència de K-Means/K-Means++, en K-Medoids el **centroide i el representant són el mateix**: ambdós són el medoide (un punt real del dataset). Per tant, `getCentroid()` i `getRepresentant()` retornen el mateix valor.

Estructura Triada

K-Medoids és força diferent de K-Means perquè usa punts reals (medoides) enlloc de centroides calculats.

Classe principal: KMedoids.java

Atributs:

```
private final DistanceCalculator dc = new DistanceCalculator();  
private final Random rnd;
```

Estructures clau: - `int[] medoidIdx`: índexs dels punts que són medoides (enlloc de còpies) - `int[] assignment`: assignació de cada punt a un cluster (0 a K-1)

Justificació

1. Per què usar índexs enlloc de còpies dels punts?

Aquesta va ser una decisió important. Els medoides són punts reals del dataset, així que teníem dues opcions:

- Opció A: Copiar els punts que són medoides
- Opció B: Guardar només els índexs d'on estan al dataset

Vam escollir l'opció B (índexs) perquè:

- Estalvia memòria: no duplica les dades
- Més coherent: el medoide apunta directament al punt del dataset original
- Accés ràpid: `data.get(medoidIdx[i])` és $O(1)$

2. Per què usar distància Manhattan enlloc d'Euclidiana?

K-Medoids tradicionalment usa distància Manhattan (L1) en comptes d'Euclidiana (L2):

Manhattan: $d(a,b) = \sum |a_i - b_i| / N$
Euclidiana: $d(a,b) = \sqrt{(\sum (a_i - b_i)^2)} / N$

Motius:

- Més robusta a outliers: Manhattan no eleva al quadrat, així que els outliers tenen menys pes
- Més eficient: no cal calcular l'arrel quadrada
- Més estable: minimitzar Manhattan amb punts reals funciona millor

3. Per què buscar el millor medoide exhaustivament?

A cada iteració, per cada cluster, provem tots els punts del cluster com a candidats a medoide i seleccionem el que minimitza la suma de distàncies. Això és costós ($O(N^2)$) però: - Garanteix trobar el millor medoide dins del cluster - És l'algoritme PAM estàndard - Per datasets petits (< 10.000 punts) és acceptable

4. Doble condició de convergència

K-Medoids para quan ni els medoides ni les assignacions canvien. Això és més estricte que K-Means (que només mira si canvien els centroides), però:

- Evita oscil·lacions: casos on medoides i assignacions canvien alhora sense trobar una solució estable
- Garanteix una millor convergència

Alternatives Descartades

Alternativa	Per què la vam descartar
Distància Euclidiana	Menys adequada per medoides. L'Euclidiana està pensada per centroides calculats com mitjanes.
Només mirar canvis d'assignació	Podria parar amb medoides subòptims. La doble condició és més segura.

Complexitat

Complexitat temporal:

Aquí és on K-Medoids paga el preu de ser més robust:

Inicialització:	$O(K)$	// Seleccionar K índexs
Assignació (per iter):	$O(N \times K \times D)$	// Igual que K-Means
Cerca millor medoide:	$O(K \times N^2 \times D)$	// Per cada cluster, provar N candidats
Per iteració:	$O(K \times N^2 \times D)$	// El terme dominant
Total:	$O(I \times K \times N^2 \times D)$	

Comparativa: - K-Means: $O(I \times N \times K \times D)$ - K-Medoids: $O(I \times K \times N^2 \times D)$

Conclusió: K-Medoids és **N vegades més lent** que K-Means. Per això només l'usem amb datasets petits o quan necessitem robustesa a outliers.

4. DistanceCalculator

Estructura Triada

Aquesta classe és la base del sistema de càlcul de distàncies entre participants.

Components principals:

Enum per tipus de variables:

```
public enum VariableKind {  
    NUMERIC,           // Variables numèriques (edat, salari...)  
    ORDINAL,           // Categòriques ordenades (satisfacció:  
    baix/mig/alt)  
    NOMINAL_SINGLE,    // Categòriques no ordenades (color preferit)  
    NOMINAL_MULTI,     // Selecció múltiple (idiomes que parles)  
    FREE_TEXT           // Text lliure (comentaris)  
}
```

Classe de metadades:

```
public static class FeatureSpec {  
    public final VariableKind kind;  
    public final List<String> ordinalOrder; // Per ORDINAL  
    public final Double numericMin, numericMax; // Per NUMERIC  
    // ... altres camps  
}
```

Mètodes principals: - distance(): distància Euclidiana (L2) - distanceManhattan(): distància Manhattan (L1)

Justificació

1. Per què un enum per tipus de variables?

Vam considerar usar Strings tipus "numeric", "ordinal", però un enum és molt millor:

- Autocomplete a l'IDE: et suggereix les opcions possibles
- Si afegim un nou tipus, el compilador ens avisa si ens hem oblidat de tractar-lo en algun switch
- No pots escriure malament un tipus per error

2. Per què FeatureSpec immutable (camps final)?

Tots els camps de FeatureSpec són final:

- Evita errors per canvis accidentals
- Un cop creat, sabem que és fix i no canviarà

3. Per què dos mètodes de distància (Euclidiana i Manhattan)?

Cada algoritme té les seves preferències:

- K-Means i K-Means++: Euclidiana (funciona bé amb mitjanes)
- K-Medoids: Manhattan (més fiable amb punts reals)

Tenim les dues implementades al mateix lloc per:

- Aprofitar el mateix codi per calcular distàncies (NUMERIC, ORDINAL, etc.)
- No repetir codi

4. Com es calculen les distàncies per cada tipus?

Cada tipus de variable té la seva fórmula:

Tipus	Fórmula	Per què
NUMERIC	$ a - b / (\max - \min)$	Normalitza per rang
ORDINAL	$ \text{pos}(a) - \text{pos}(b) / (m - 1)$	Respecta l'ordre
NOMINAL_SINGLE	$a == b ? 0 : 1$	Binari: igual o diferent
NOMINAL_MULTI	$1 - \text{Jaccard}(a, b)$	Mesura solapament de conjunts
FREE_TEXT	Levenshtein normalitzat	Distància d'edició

Alternatives Descartades

Alternativa	Per què la vam descartar
Strings per tipus	Sense validació en compilació. Propenso a typos i errors.
Herència (NumericSpec, OrdinalSpec...)	Massa complicat per al que necessitem. A més, dificulta convertir les dades a JSON.
List<FeatureSpec> enlloc d'array	Gasta recursos innecessaris. El nombre de dimensions és sempre fix (D).
Mètode únic amb paràmetre metric	Menys clar. Més propenso a errors d'ús.

Complexitat

Complexitat temporal:

Depèn del tipus de variables:

`distance(a, b):` $O(D)$ `// Recorre D dimensions`

Per cada dimensió:

NUMERIC:	$O(1)$	// Resta i divisió
ORDINAL:	$O(M)$	// Buscar posició (M opcions)
NOMINAL_SINGLE:	$O(1)$	// Comparació
NOMINAL_MULTI:	$O(S)$	// Jaccard (S seleccions)
FREE_TEXT:	$O(L^2)$	// Levenshtein (L longitud)
Cas pitjor:	$O(D \times L^2)$	// Si totes són FREE_TEXT
Cas típic:	$O(D)$	// Poques FREE_TEXT

5. ClusterEvaluator

Estructura Triada

Aquesta classe calcula el coeficient de Silhouette per avaluar la qualitat del clustering.

Classe principal: ClusterEvaluator.java

Atribut:

```
private final DistanceCalculator dc = new DistanceCalculator();
```

Mètodes públics: - silhouetteScore(): score global per tot el clustering -
silhouettePerCluster(): score per cada cluster individual

Mètodes privats: - silhouettePoint(): càlcul per un punt - Mètodes auxiliars per calcular a(i) i b(i)

Justificació

1. Per què una classe sense estat ?

ClusterEvaluator no guarda res entre crides, cada vegada és independent:

- Es pot usar des de múltiples fils alhora sense problemes
- Sempre dona el mateix resultat amb les mateixes dades
- Més fàcil de provar

2. Per què dos mètodes públics (global i per cluster)?

Hem volgut donar flexibilitat:

- silhouetteScore(): per comparar diferents algoritmes o valors de K
- silhouettePerCluster(): per veure quins clusters estan mal formats

Els dos usen silhouettePoint() internament per no repetir codi.

3. Com funciona Silhouette?

Per cada punt i:

a(i) = distància mitjana als punts del mateix cluster (cohesió)

b(i) = distància mitjana al cluster més proper (separació)

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

Interpretació: - $s(i) \approx 1$: punt ben assignat - $s(i) \approx 0$: punt al límit entre clusters - $s(i) < 0$: punt possiblement mal assignat

4. Per què Silhouette i no altres mètriques?

Silhouette és:

- Intuitiu: combina què tan junts estan els punts del mateix cluster i què tan separats estan d'altres clusters
- No supervisat: no necessita saber les etiquetes correctes d'entrada
- Comparable: serveix per comparar diferents valors de K

Alternatives Descartades

Alternativa	Per què la vam descartar
Emmagatzemar resultats	Malbaratament de memòria. Cada avaluació és independent.
Mètode únic amb flag	Menys clar. Millor tenir dues interfícies separades.

Complexitat

Complexitat temporal:

Aquí està el problema: Silhouette és costós.

`silhouetteScore()`:

Per cada punt (N total):

`a(i):  $O(N \times D)$` // Distància a punts del mateix cluster

`b(i):  $O(K \times N \times D)$` // Distància a K clusters

Total: $O(N \times K \times N \times D) = O(K \times N^2 \times D)$

Important: És $O(N^2)$, molt costós per datasets grans.

Per al nostre projecte ($N \approx 1.000$), és acceptable. Per $N > 10.000$, caldria considerar aproximacions.

6. Kluster

Estructura Triada

Aquesta classe representa un cluster amb el seu centroide i membres.

Classe principal: `Kluster.java`

Atributs:

```
private String[] centroid;  
private List<String[]> members;
```

Mètodes clau: - `addMember()`: afegir participant - `recomputeCentroid()`: recalculer centroide segons tipus de variables - `getCentroid()`, `getMembers()`: getters amb còpia defensiva - `getRepresentant()`: obtenir el participant real més proper al centroide

Justificació

1. Per què separar centroide i membres?

Podríem haver-ho posat tot junt, però separar-ho té sentit:

- El centroide és el punt representatiu (pot ser calculat o un punt real)
- Els membres són els participants assignats
- Així la mateixa classe serveix tant per K-Means (centroides calculats) com per K-Medoids (medoides reals)

2. Per què `ArrayList` per membres?

Necessitem una llista dinàmica perquè no sabem quants membres tindrà cada cluster:

- `ArrayList`: accés ràpid per índex i creix automàticament
- `LinkedList`: descartada, accés més lent
- `HashSet`: descartada, compara per referència i no ens cal unicitat automàtica

3. Per què `recomputeCentroid()` retorna `boolean`?

El mètode recalcula el centroide i retorna `true` si ha canviat: - Útil per detectar convergència a K-Means - Evita comparacions externes

4. Com es calculen els centroides per diferents tipus?

Cada tipus té la seva agregació:

Tipus	Agregador	Per què
NUMERIC	Mitjana	Minimitza $\sum (x_i - \mu)^2$
ORDINAL	Mediana	Minimitza $\sum x_i - c $ preservant ordre
NOMINAL	Moda	Valor més freqüent
MULTI	Intersecció majoritària	Maximitza Jaccard

TEXT	Centroide textual	Minimitza Levenshtein mitjana
------	-------------------	-------------------------------

5. Per què un mètode `getRepresentant()` a més de `getCentroid()`?

Aquesta és una funcionalitat molt important que vam afegir. El problema és que en K-Means i K-Means++, el centroide és un punt calculat (mitjanes, modes), no necessàriament un participant real.

Per això vam implementar `getRepresentant()` que:

- Per K-Means/K-Means++: retorna el participant real més proper al centroide calculat
- Per K-Medoids: retorna directament el medoide (que ja és un punt real)

Això permet:

- Mostrar a l'usuari un participant real que representa el cluster
- Tenir un exemple concret i fàcil d'interpretar del cluster
- Evitar confusions amb valors decimals o combinacions impossibles

Alternatives Descartades

Alternativa	Per què la vam descartar
Array String[][] per membres	Mida fixa. Malbaratament si un cluster té pocs membres.
HashSet<String[]>	Compara per referència, no per contingut. No funciona.
Guardar índexs enlloc de punts	Menys intuïtiu. Requereix passar el dataset a cada operació.
Centroide i membres junts	Confús. El rol de cada un no estaria clar.
Només <code>getCentroid()</code> sense <code>getRepresentant()</code>	Els centroides calculats poden tenir valors impossible (27.33 anys, per exemple). Cal un participant real.
<code>getRepresentant()</code> sense paràmetres	Necessitem <code>DistanceCalculator</code> i specs per calcular distàncies correctament segons tipus de variables.

Complexitat

Complexitat temporal:

```
addMember():           O(1) amortitzat // ArrayList.add()
recomputeCentroid():   O(M × D)         // M membres, D dimensions
getRepresentant():     O(M × D)         // Cerca del punt més proper
```

Per tipus (`recomputeCentroid()`):

```
NUMERIC:               O(M)             // Suma i divisió
```

ORDINAL:	$O(M \log M)$	// Sort per mediana
NOMINAL:	$O(M)$	// Comptatge
TEXT:	$O(M \times L)$	// Comparacions text

Per `getRepresentant()`:

Cerca lineal:	$O(M)$	// Iterar tots els membres
Per membre:	$O(D)$	// Calcular distància
Total:	$O(M \times D)$	// M membres $\times$ D dimensions

Nota sobre `getRepresentant()`:

Tot i que té cost $O(M \times D)$, només es crida un cop al final del clustering (no a cada iteració), així que l'impacte en el rendiment total és negligible.

7. Taula Comparativa

Aquesta taula resumeix les diferències entre els tres algoritmes implementats:

Criteri	K-Means	K-Means++	K-Medoids
Complexitat temporal	$O(I \times N \times K \times D)$	$O(I \times N \times K \times D)$	$O(I \times K \times N^2 \times D)$
Inicialització	Aleatòria $O(K)$	Intel·ligent $O(K^2 \times N \times D)$	Aleatòria $O(K)$
Distància usada	Euclidiana (L2)	Euclidiana (L2)	Manhattan (L1)
Centroides	Calculats (mitjanes)	Calculats (mitjanes)	Punts reals (medoides)
Quan usar-lo	Datasets grans, necessitem velocitat	Cas general (balanced)	Datasets petits, outliers importants

Llegenda: - **I** = nombre d'iteracions fins convergència - **N** = nombre de participants - **K** = nombre de clusters - **D** = nombre de dimensions (preguntes)

Recomanacions d'ús:

- **K-Means:** Si tens milers de participants i necessites velocitat
- **K-Means++:** És la teva millor opció en general. Millor qualitat que K-Means amb cost similar
- **K-Medoids:** Només si tens outliers problemàtics i un dataset petit (< 10.000 punts)